

Spatial Data Science

Graduate – Elective (MSDS/MSCS/PhDCS)



Remote Sensing
&
Spatial Analytics

Dr. sc. Muhammad Adnan Siddique
Assistant Professor
Information Technology University - ITU

Everything is related to everything else,
but near things are more related than
distant things.

W. Tobler

Spatial Data - I

Introduction to spatial data structures, their uniqueness, and how to manipulate them

Spatial Data

- a feature is one measured trait pertaining to an observation (a column), and a sample is one set of measurements (a row).
- The geometry of the country's boundary is stored in the geometry column, and is encoded using a special class in Python that is used to represent geometric objects.

```
gt_polygons = geopandas.read_file(  
    "../data/countries/countries_clean.gpkg"  
)
```

```
gt_polygons.head()
```

	ADMIN	geometry
Sample → 0	Indonesia	MULTIPOLYGON (((13102705.696 463877.598, 13102...
1	Malaysia	MULTIPOLYGON (((13102705.696 463877.598, 13101...
Feature → 2	Chile	MULTIPOLYGON (((-7737827.685 -1979875.500, -77...
3	Bolivia	POLYGON ((-7737827.685 -1979875.500, -7737828....



Spatial Data

- a feature is one measured trait pertaining to an observation (a column), and a sample is one set of measurements (a row).
- This geographic table is an instance of the `geopandas.GeoDataFrame` object, used throughout Python's ecosystem to represent geographic data.

```
gt_polygons = geopandas.read_file(  
    "../data/countries/countries_clean.gpkg"  
)
```

```
gt_polygons.head()
```

	ADMIN	geometry
Sample → 0	Indonesia	MULTIPOLYGON (((13102705.696 463877.598, 13102...
1	Malaysia	MULTIPOLYGON (((13102705.696 463877.598, 13101...
Feature → 2	Chile	MULTIPOLYGON (((-7737827.685 -1979875.500, -77...
3	Bolivia	POLYGON ((-7737827.685 -1979875.500, -7737828....



Geographic tables

- Geographic tables store geographic information as an additional column. But, how is this information encoded?

```
type(gt_polygons.geometry[0])
```

```
shapely.geometry.multipolygon.MultiPolygon
```

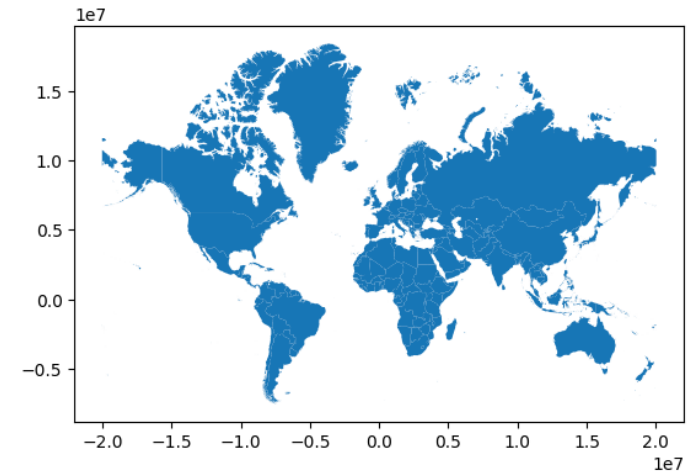
- In `geopandas` (as well as other packages representing geographic data), the `geometry` column has special traits which a “normal” column, such as `ADMIN`, does not.



Geometry column

- For example, when we plot the dataframe, the `geometry` column is used as the main shape to use in the plot, as shown in figure below.

```
gt_polygons.plot();
```



- Since the `geometry` column is special, there are special functions to adjust the geometry.
 - For example, if we wanted to represent each country using its *centroid*, a point in the middle of the shape, then we must take care to make sure that a new geometry column was set properly using the `set_geometry()` method.



Setting geometry

- Let us make a map of both the boundary and the centroid of a country.
 - First, to compute the centroid, we can use the `gt_polygons.geometry.centroid` property.
 - This gives us the point that minimizes the average distance from all other points on the boundary of the shape.
 - Storing that back to a column, called centroid:

```
gt_polygons["centroid"] = gt_polygons.geometry.centroid
```

```
gt_polygons.head()
```

```
# Plot centroids
ax =
gt_polygons.set_geometry("centroid").plot("ADMIN", markersize=5)
# Plot polygons without color filling
gt_polygons.plot(
    "ADMIN", ax=ax, facecolor="none",
    edgecolor="k", linewidth=0.2
);
```

	ADMIN	geometry	centroid
0	Indonesia	MULTIPOLYGON (((13102705.696 463877.598, 13102...	POINT (13055431.810 -248921.141)
1	Malaysia	MULTIPOLYGON (((13102705.696 463877.598, 13101...	POINT (12211696.493 422897.505)
2	Chile	MULTIPOLYGON (((-7737827.685 -1979875.500, -77...	POINT (-7959811.948 -4915458.802)
3	Bolivia	POLYGON ((-7737827.685 -1979875.500, -7737828....	POINT (-7200010.945 -1894653.148)

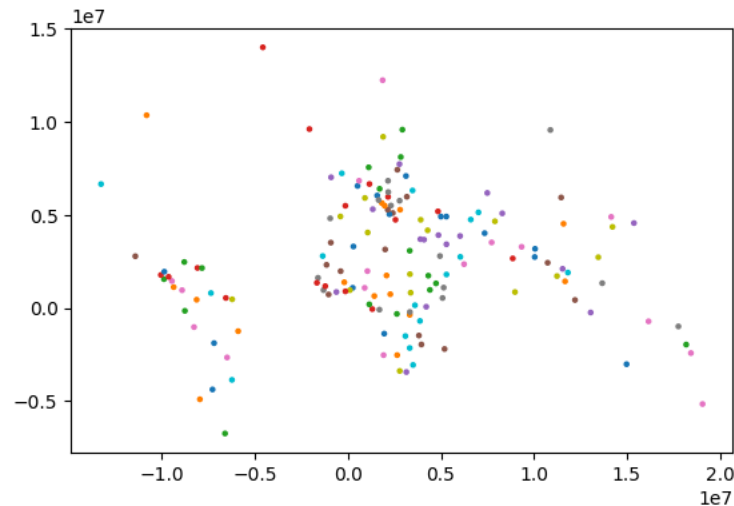


Setting geometry

- Let us make a map of both the boundary and the centroid of a country.
 - First, to compute the centroid, we can use the `gt_polygons.geometry.centroid` property.
 - This gives us the point that minimizes the average distance from all other points on the boundary of the shape.
 - Storing that back to a column, called centroid:

```
gt_polygons["centroid"] = gt_polygons.geometry.centroid
```

```
gt_polygons.head()
```



	ADMIN	geometry	centroid
0	Indonesia	MULTIPOLYGON (((13102705.696 463877.598, 13102...	POINT (13055431.810 -248921.141)
1	Malaysia	MULTIPOLYGON (((13102705.696 463877.598, 13101...	POINT (12211696.493 422897.505)
2	Chile	MULTIPOLYGON (((-7737827.685 -1979875.500, -77...	POINT (-7959811.948 -4915458.802)
3	Bolivia	POLYGON ((-7737827.685 -1979875.500, -7737828....	POINT (-7200010.945 -1894653.148)

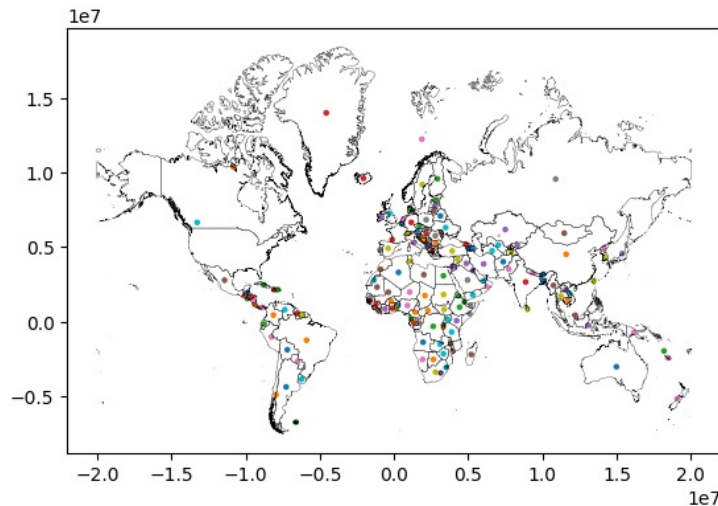


Setting geometry

- Let us make a map of both the boundary and the centroid of a country.
 - First, to compute the centroid, we can use the `gt_polygons.geometry.centroid` property.
 - This gives us the point that minimizes the average distance from all other points on the boundary of the shape.
 - Storing that back to a column, called centroid:

```
gt_polygons["centroid"] = gt_polygons.geometry.centroid
```

```
gt_polygons.head()
```



	ADMIN	geometry	centroid
0	Indonesia	MULTIPOLYGON (((13102705.696 463877.598, 13102...	POINT (13055431.810 -248921.141)
1	Malaysia	MULTIPOLYGON (((13102705.696 463877.598, 13101...	POINT (12211696.493 422897.505)
2	Chile	MULTIPOLYGON (((-7737827.685 -1979875.500, -77...	POINT (-7959811.948 -4915458.802)
3	Bolivia	POLYGON ((-7737827.685 -1979875.500, -7737828....	POINT (-7200010.945 -1894653.148)



Geometric Objects

What are the basic geometric objects and how are they constructed?

Remarks

- Nearly any kind of geographic object can be represented in one (or more) geometry column(s).
- Defining too many different "abstractions" for geometry will lead to complexity.
- The Open Geospatial Consortium (OGC) has defined a set of "abstract" types that can be used to define any kind of geometry.
- Let's look at those!



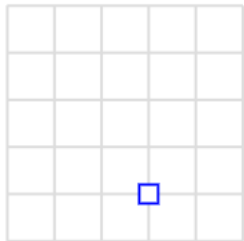
ISO 19125-1—the “simple features”

- This specification, codified in ISO 19125-1—the “simple features” specification—defines the formal relationships between these types:
- a `Point` is a zero-dimensional location with an x and y coordinate;
- a `LineString` is a path composed of a set of more than one `Point`, and
- a `Polygon` is a surface that has at least one `LineString` that starts and stops with the same coordinate.
- All of these types also have `Multi` variants that indicate a collection of multiple geometries of the same type.

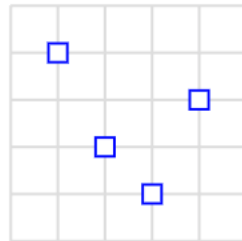


ISO 19125-1—the “simple features”

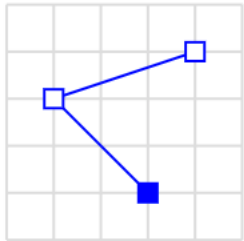
- All geospatial vector data can be described by a combining a set of fundamental geometric objects: **points**, **lines**, and **polygons** are the basic ingredients of working with spatial data.



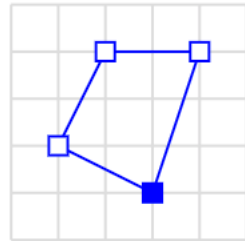
Point



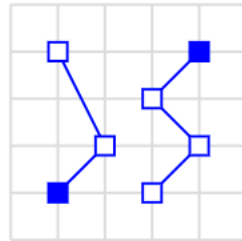
MultiPoint



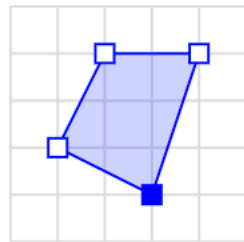
Linestring



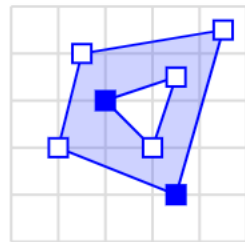
LinearRing



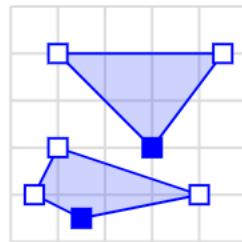
MultiLinestring



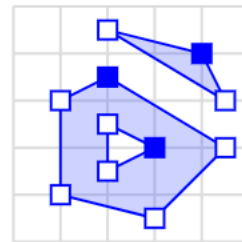
Polygon



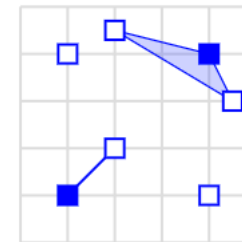
Polygon



MultiPolygon



MultiPolygon



GeometryCollection



Example

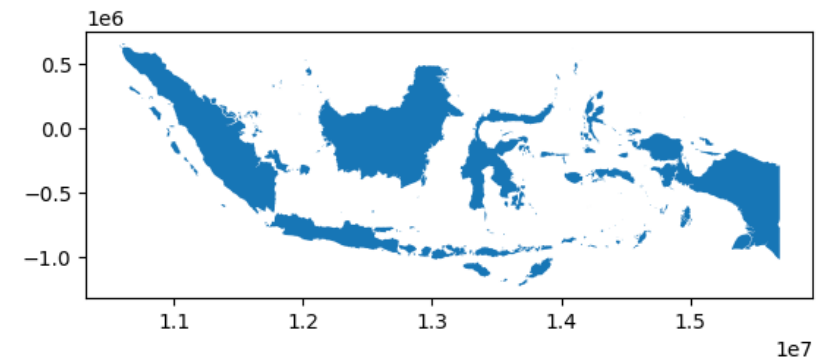
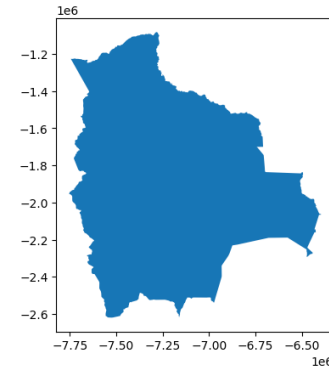
- Bolivia is represented as a single **Polygon**, while Indonesia is a **Multipolygon**:

```
gt_polygons.query('ADMIN == "Bolivia"')
```

ADMIN	geometry	centroid
3 Bolivia	POLYGON ((-7737827.685 -1979875.500, -7737828....	POINT (-7200010.945 -1894653.148)

```
gt_polygons.query('ADMIN == "Indonesia"')
```

ADMIN	geometry	centroid
0 Indonesia	MULTIPOLYGON (((13102705.696 463877.598, 13102...	POINT (13055431.810 -248921.141)



Points

- These are the simplest type of feature in that they do not have any dimension, only a pair of coordinates attached to them.
- This means that points can sometimes be stored in a non-geographic table, simply using one column for each coordinate.

```
gt_points = pandas.read_csv("../data/tokyo/tokyo_clean.csv")
```

```
type(gt_points)
```



```
pandas.core.frame.DataFrame
```



Points

- These are the simplest type of feature in that they do not have any dimension, only a pair of coordinates attached to them.
- This means that points can sometimes be stored in a non-geographic table, simply using one column for each coordinate.

```
gt_points.head()
```

	user_id	longitude	latitude	date_taken	photo/video_page_url	x	y
0	10727420@N00	139.700499	35.674000	2010-04-09 17:26:25.0	http://www.flickr.com/photos/10727420@N00/4545...	1.555139e+07	856e+06
1	8819274@N04	139.766521	35.709095	2007-02-10 16:08:40.0	http://www.flickr.com/photos/8819274@N04/26503...	1.555874e+07	0667e+06
2	62068690@N00	139.765632	35.694482	2008-12-21 15:45:31.0	http://www.flickr.com/photos/62068690@N00/3125...	1.555864e+07	8664e+06

no geometry column



Points

- To make the most of them, it is convenient to convert them into `GeoDataFrame` tables. There are two steps involved in this process. First, we turn the raw coordinates into geometries.
- Second, we create a `GeoDataFrame` object using these geometries:

```
pt_geoms = geopandas.points_from_xy(  
    x=gt_points["longitude"],  
    y=gt_points["latitude"],  
    # x,y are Earth longitude & latitude  
    crs="EPSG:4326",  
)
```

✓ 0.1s

```
gt_points = geopandas.GeoDataFrame(gt_points, geometry=pt_geoms)
```

✓ 0.5s



Points

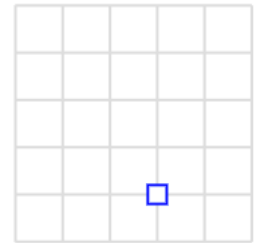
- And now `gt_points` looks and feels exactly like the one of countries we have seen before, with the difference the geometry column stores `POINT` geometries:

gt_points.head()								
✓ 0.6s								
Python								
	user_id	longitude	latitude	date_taken	photo/video_page_url	x	y	geometry
0	10727420@N00	139.700499	35.674000	2010-04-09 17:26:25.0	http://www.flickr.com/photos/10727420@N00/4545...	1.555139e+07	4.255856e+06	POINT (139.70050 35.67400)
1	8819274@N04	139.766521	35.709095	2007-02-10 16:08:40.0	http://www.flickr.com/photos/8819274@N04/26503...	1.555874e+07	4.260667e+06	POINT (139.76652 35.70909)
2	62068690@N00	139.765632	35.694482	2008-12-21 15:45:31.0	http://www.flickr.com/photos/62068690@N00/3125...	1.555864e+07	4.258664e+06	POINT (139.76563 35.69448)



Points

- Point geometries represent a singular point (in two- or three-dimensional Euclidean space). Points are defined by a single coordinate tuple.

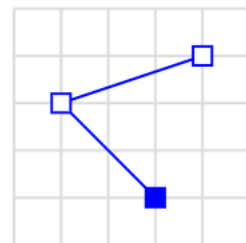


Point

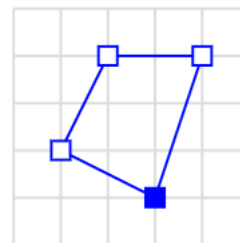


Lines

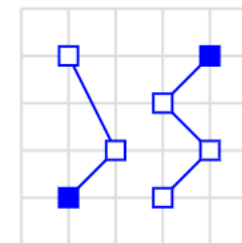
- `LineString` geometries (and the related `LinearRings`) represent lines. They are defined by a sequence of points.
- By connecting the points in order, a line is formed, if the first and last point are the same, a linear ring. Consequently, to describe a `LineString`, at least two points are necessary, for a `LinearRing` at least three (first and last being identical).



LineString



LinearRing

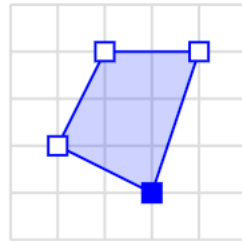


MultiLineString

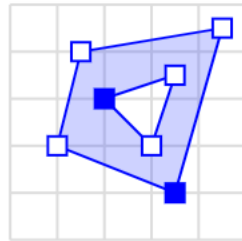


Polygons

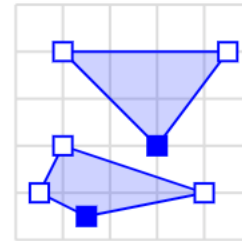
- **Polygon** geometries represent an area.
- A polygon is defined by exactly one **LinearRing** as its circumference, and any number of additional **LinearRings** representing holes that are cut out.
- As such, at minimum, three coordinate tuples are needed to define a **Polygon** (but it becomes more difficult quickly, as, naturally, the coordinates that define the holes have to lie within the exterior **LinearRing**, and also must not intersect each other).



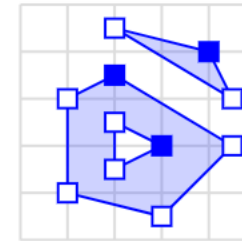
Polygon



Polygon



MultiPolygon

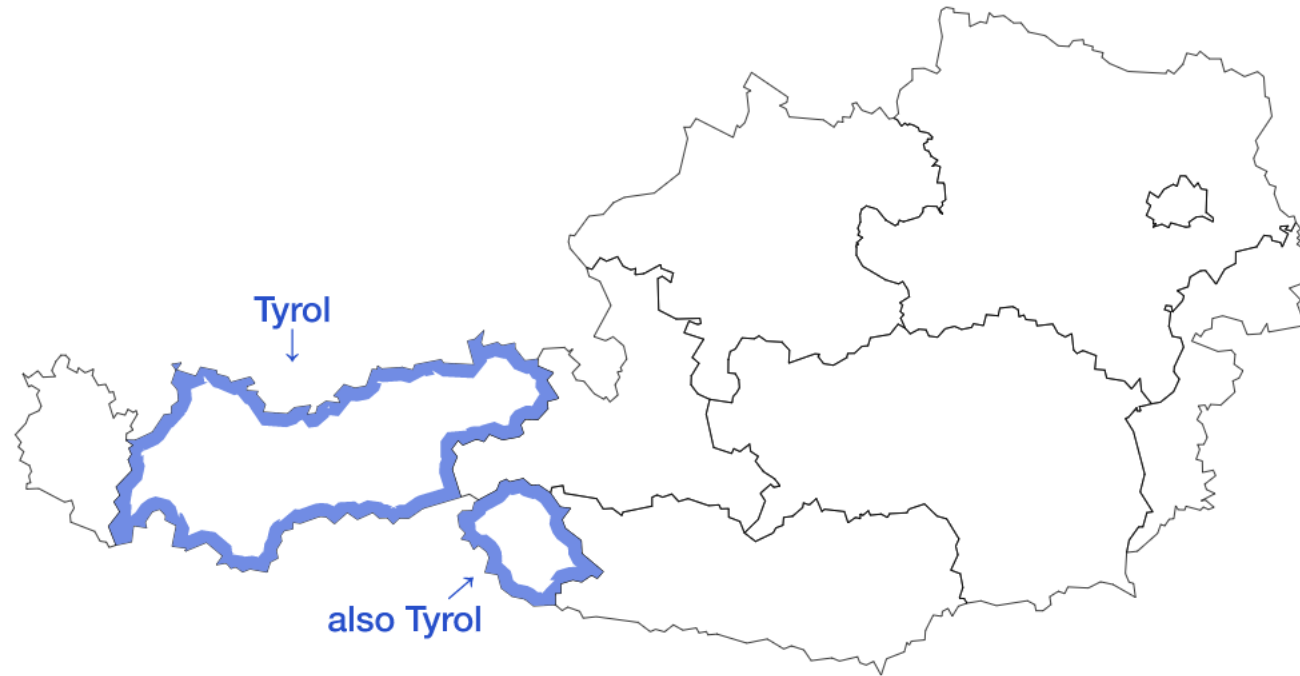


MultiPolygon



Collections

- All of the fundamental geometric types can be combined to represent more complex geographic features, such as, for instance, administrative areas that consist of multiple discontinuous regions:



Tyrol, a federal state of Austria, is a **MultiPolygon**. (Data: [Statistics Austria](#))



Summary

- `MultiPoint` geometries represent collections of points.
- `MultiLineString` geometries represent collections of lines.
- `MultiPolygon` geometries represent collections of polygons.
- `GeometryCollection` geometries are collections of points, lines, and polygons, as well as multi-points, multi-lines, and multi-polygons.



Geometric Objects

Using shapely to create, access, manipulate and analyse geometric objects

Shapely

- **Shapely** can perform many useful operations on geometries, and provides a range of attributes of geometries. For instance, you can:
 - create lines and polygons from a set of points
 - calculate the area, length, perimeter, etc., of geometries
 - perform geometric operations on a set of geometries, for instance, merging (union), subtracting (difference), or calculating the distance between geometries.
 - query the relationship between geometries, i.e., find out whether two geometries intersect, touch, cross, etc.



Coordinate tuples

- Geometric objects are described and constructed by coordinate tuples.
- Coordinate tuples typically are either two-dimensional (x, y) , or three-dimensional (x, y, z) .

Tuples

A **tuple** is a Python data structure that consists of a number of values separated by commas. Coordinate pairs are often represented as a tuple. For example, $(60.192059, 24.945831)$ is a tuple with two values, $(1,)$ a tuple with one value (note the comma that distinguishes a one-value tuple from a mathematical expression in parentheses), and `tuple([1, 2, 3])` converts ('casts') a **list** into a **tuple**.



Coordinate tuples

- Geometric objects are described and constructed by coordinate tuples.
- Coordinate tuples typically are either two-dimensional (x, y) , or three-dimensional (x, y, z) .

Tuples belong to the **sequence data types** in Python. Other sequence data types are lists and ranges. Tuples have many similarities with lists and ranges, but they are often used for different purposes.

The main difference between tuples and lists is that tuples are **immutable**, which means that the contents of a tuple cannot be altered (while lists are mutable; you can, for example, add and remove values from lists).



Points

- Creating a point geometry object is easy: simply pass coordinates (x, y, and possibly z) to its *constructor*.

```
# Import `shapely.geometry.Point` class
from shapely.geometry import Point

# Create `Point` objects:
point1 = Point(2.2, 4.2)
point2 = Point(7.2, -25.1)
point3 = Point(9.26, -2.456)
point4_3D = Point(9.26, -2.456, 0.57)
```

point1



```
print(point1)
print(point4_3D)
```

```
POINT (2.2 4.2)
POINT Z (9.26 -2.456 0.57)
```

This text representation is in [‘Well-Known Text’ \(WKT\) format](#), a standard set forward in the [Open Geospatial Consortium’s \(OGC\) Simple Feature Access](#). This includes the additional letter ‘Z’ that marks the three-dimensional version of a geometry (e.g., our point point4_3D).



Point

- Let's also check the data type of a point:

```
type(point1)
```

```
shapely.geometry.point.Point
```

- We can see that the type of the point is a `shapely.geometry.point.Point` (which is equivalent to `shapely.geometry.Point`, the class we used to instantiate the point object).
- Under the hood, shapely uses `GEOS` to handle geometry objects. `GEOS` is a C++ library (much faster than Python code), and is one of the fundamental pillars of the open source GIS world, powering geospatial processing for many projects, including QGIS.



Point properties and methods

- Points and other shapely geometry objects have useful built-in properties and methods.
 - Using the available attributes, we can for example extract the coordinate values of a Point and calculate the Euclidian distance between points.
- The `geom_type` property contains information about the geometry type of a `shapely` geometry:

```
point1.geom_type
```

```
'Point'
```



Point properties and methods

- Points and other shapely geometry objects have useful built-in properties and methods.
 - Using the available attributes, we can for example extract the coordinate values of a Point and calculate the Euclidian distance between points.
- `coords` is a `shapely.coords.CoordinateSequence`. It is an **Iterator**.
 - we can simply convert it into a list of the (one pair of) coordinates:

```
# Get coordinate tuple(s)  
list(point1.coords)
```

```
[(2.2, 4.2)]
```



Point properties and methods

- Points and other shapely geometry objects have useful built-in properties and methods.
 - Using the available attributes, we can for example extract the coordinate values of a Point and calculate the Euclidian distance between points.
- However, since points, by definition, only contain one coordinate tuple, `shapely.geometry.Point` have properties to directly access its coordinate values:

the properties `x`, `y`, and (possibly) `z`, which are basic `float` type decimal numbers.

```
# Read x, y, and z
x = point4_3D.x
y = point4_3D.y
z = point4_3D.z

print(x, y, z)
```

```
9.26 -2.456 0.57
```



Lines

- Creating `LineString` objects is similar to creating `Points`. Instead of a single coordinate tuple, we pass a list of coordinate tuples, or a list of points, that make up the line:

```
# import the LineString class
from shapely.geometry import LineString

# Create a LineString from our Point objects
line = LineString([point1, point2, point3])
```

```
# Check geometry type of the line object
line.geom_type
```

```
'LineString'
```

```
line
```



```
print(line)
```

```
LINESTRING (2.2 4.2, 7.2 -25.1, 9.26 -2.456)
```



LineString properties and methods

- Linear geometries in their shapely representations (`LineString`, `LinearRing`, `MultiLineString`) have a variety of properties and methods that expose useful functionality.

```
# Get coordinate tuples  
list(line.coords)
```

```
[(2.2, 4.2), (7.2, -25.1), (9.26, -2.456)]
```

```
# Obtain x and y coordinates  
xcoords = list(line.xy[0])  
ycoords = list(line.xy[1])
```

```
print(xcoords)  
print(ycoords)
```

```
[2.2, 7.2, 9.26]  
[4.2, -25.1, -2.456]
```

⚠ 3D-LineStrings

Note that the `xy` property of shapely geometries does not return `z` values for three-dimensional geometries.



LineString properties and methods

- Linear geometries in their shapely representations (`LineString`, `LinearRing`, `MultiLineString`) have a variety of properties and methods that expose useful functionality.

```
# Get the length of the line  
line_length = line.length  
print(f"Length of our line: {line_length:.1f} units")
```

```
Length of our line: 52.5 units
```

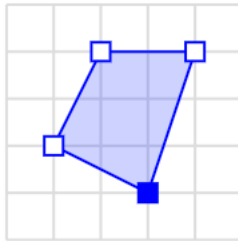
```
# Get the centre point of the line  
print(line.centroid)
```

```
POINT (6.229961354035622 -11.892411157572392)
```

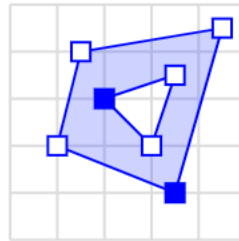


Polygon

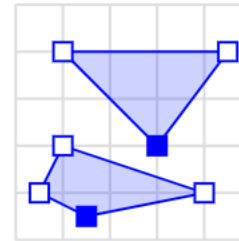
- Consequently, the `shapely.geometry.Polygon` constructor function accepts two parameter:
 - the first one, `shell`, is a list of coordinate tuples, a list of points, or a `LinearRing`, and will form the outer hull of the new polygon.
 - The second, optional, parameter `holes` can be a list of holes to cut out of `shell` (the items in the list can be the same data types as `shell`).



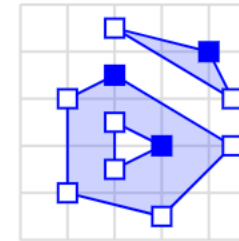
Polygon



Polygon



MultiPolygon



MultiPolygon



Polygon

- For now, let's create a simple polygon without any holes.

```
from shapely.geometry import Polygon

# Create a Polygon from the coordinates
polygon1 = Polygon([(2.2, 4.2), (7.2, -25.1), (9.26, -2.456)])
```



We can also construct the polygon directly from a list of points:

```
polygon2 = Polygon([point1, point2, point3])
```

```
# Data type
type(polygon1)
```

```
shapely.geometry.polygon.Polygon
```

```
# Geometry type
polygon1.geom_type
```

```
'Polygon'
```

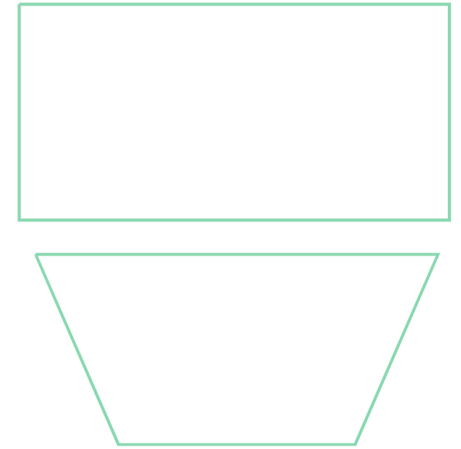


Polygon

- Let's still see how to create a polygon with a hole:

```
# define the exterior
outer = LinearRing([(-180, 90), (-180, -90), (180, -90), (180, 90)])

# define a hole:
hole = LinearRing([(-170, 80), (-100, -80), (100, -80), (170, 80)])
```



```
polygon_without_hole = Polygon(outer)
polygon_without_hole
```



```
polygon_with_hole = Polygon(outer, [hole])
polygon_with_hole
```



Polygon properties and methods

- A few examples:

```
print(f"Polygon centroid: {polygon_with_hole.centroid}")
print(f"Polygon area: {polygon_with_hole.area}")
print(f"Polygon bounding box: {polygon_with_hole.bounds}")
print(f"Polygon exterior ring: {polygon_with_hole.exterior}")
print(f"Polygon circumference: {polygon_with_hole.exterior.length}")
```

```
Polygon centroid: POINT (0 -13.827160493827162)
Polygon area: 21600.0
Polygon bounding box: (-180.0, -90.0, 180.0, 90.0)
Polygon exterior ring: LINEARRING (-180 90, -180 -90, 180 -90, 180 90, -180 90)
Polygon circumference: 1080.0
```



Geometry collections

- In shapely, collections of points are implemented as `MultiPoint` geometries, collections of lines as `MultiLineString` geometries, and collections of polygons as `MultiPolygon` geometries.

```
from shapely.geometry import MultiPoint, MultiLineString, MultiPolygon

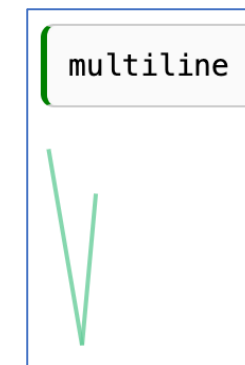
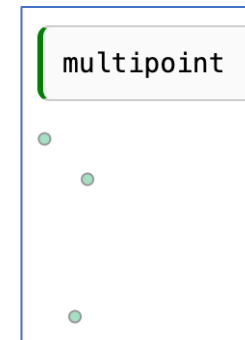
# Create a MultiPoint object of our points 1,2 and 3
multipoint = MultiPoint([point1, point2, point3])

# We can also create a MultiLineString with two lines
line1 = LineString([point1, point2])
line2 = LineString([point2, point3])
multiline = MultiLineString([line1, line2])

print(multipoint)
print(multiline)
```

```
MULTIPOINT (2.2 4.2, 7.2 -25.1, 9.26 -2.456)
```

```
MULTILINESTRING ((2.2 4.2, 7.2 -25.1), (7.2 -25.1, 9.26 -2.456))
```

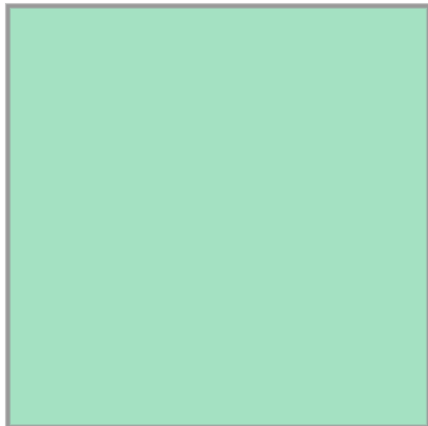


Geometry collections

- **MultiPolygons** are constructed in a similar manner. Let's create a bounding box for 'the world' by combining two separate polygons that represent the western and eastern hemispheres.

```
# Let's create the exterior of the western part of the world  
western_hemisphere = Polygon([(-180, 90), (-180, -90), (0, -90), (0, 90)])  
print(western_hemisphere)  
western_hemisphere
```

```
POLYGON ((-180 90, -180 -90, 0 -90, 0 90, -180 90))
```



Geometry collections

- **MultiPolygons** are constructed in a similar manner. Let's create a bounding box for 'the world' by combining two separate polygons that represent the western and eastern hemispheres.

```
from shapely.geometry import box
min_x = 0
max_x = 180
min_y = -90
max_y = 90

eastern_hemisphere = box(min_x, min_y, max_x, max_y)

print(eastern_hemisphere)
eastern_hemisphere
```

```
POLYGON ((180 -90, 180 90, 0 90, 0 -90, 180 -90))
```

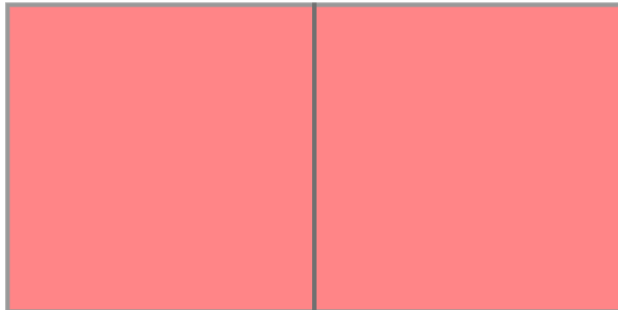


Geometry collections

- **MultiPolygons** are constructed in a similar manner. Let's create a bounding box for 'the world' by combining two separate polygons that represent the western and eastern hemispheres.

```
# Let's create our MultiPolygon.  
# Pass multiple Polygon objects as a list  
multipolygon = MultiPolygon([western_hemisphere, eastern_hemisphere])  
  
print(multipolygon)  
multipolygon
```

```
MULTIPOLYGON (((-180 90, -180 -90, 0 -90, 0 90, -180 90)), ((180 -90, 180 90,
```



Convex hull

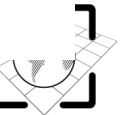
- A 'convex hull' refers to the smallest possible convex polygon that can contain a geometry or a set of geometries. Alongside bounding boxes, convex hulls are useful to describe the extent of data sets.

```
# Convex Hull  
multipoint.convex_hull
```

✓ 0.6s



```
# Envelope (smallest rectangular polygon around a geometry/set of geometries):  
multipoint.envelope
```



Validity of geometries

- Each shapely geometry has a built-in check that can be of great help, for instance, finding topological errors:

```
print(f"Is polygon valid?: {polygon_with_hole.is_valid}")
```

```
Is polygon valid?: True
```



Remarks

- There are many properties and methods for different classes just discussed.
- Please refer to the [Shapely](https://shapely.readthedocs.io/en/stable/manual.html#general-attributes-and-methods) documentation:
 - <https://shapely.readthedocs.io/en/stable/manual.html#general-attributes-and-methods>

